

Interface Gráfica Integrada ao ROS para Telemetria e Parametrização de Robô Autônomo

Rafael Corrêa Zart

Orientador: Kleyton Hoffmann

Universidade do Oeste de Santa Catarina - Unoesc

ACET - Área de Ciências Exatas e Tecnológicas

Curso de Engenharia de Computação - Campus de Joaçaba/SC

Resumo—¹ Este trabalho propõe o desenvolvimento de uma interface gráfica para telemetria e parametrização de robôs autônomos, integrada ao *Robot Operating System* (ROS) por meio do protocolo ROSBridge. Utilizando a biblioteca Streamlit, a interface foi projetada para oferecer interação intuitiva, com visualização em tempo real de velocidade, posição e mapas 2D gerados por algoritmos SLAM (*Simultaneous Localization and Mapping*), além de possibilitar ajustes de parâmetros operacionais e controle manual do robô. A metodologia empregada estrutura-se em três fases distintas: a caracterização do *hardware*, o desenvolvimento modular e a validação em ambientes simulados (RViz) e reais. O desenvolvimento modular encontra-se concluído, com o módulo de visualização e de controle implementados. *Scripts* de comunicação via ROSBridge estão operacionais e integrados à interface e funcionalidades como o mapeamento 2D com LiDAR (Light Detection and Ranging) e a implementação de *waypoints* estão funcionando. Funcionalidades planejadas na primeira fase de desenvolvimento do trabalho de conclusão de curso, como plotar mapa 2D com nuvem de pontos e navegação autônoma por waypoints foram implementadas com sucesso, testes também foram realizados para validar a precisão de posicionamento no mapa e a eficácia da interface. O sistema demonstra potencial para aplicações em ambientes internos e semi-estruturados, contribuindo para a integração eficiente entre interfaces gráficas e sistemas robóticos.

Palavras chave—aplicativo, app, interface gráfica, LiDAR, robô autônomo, ROSBridge, Streamlit, web.

Abstract—This work proposes the development of a graphical interface for telemetry and parameterization of autonomous robots, integrated with the *Robot Operating System* (ROS) through the ROSBridge protocol. Using the Streamlit library, the interface was designed to offer intuitive interaction, with real-time visualization of velocity, position, and 2D maps generated by SLAM (*Simultaneous Localization and Mapping*) algorithms, in addition to enabling adjustments of operational parameters and manual control of the robot. The methodology employed is structured in three distinct phases: *hardware* characterization, modular development, and validation in simulated (RViz) and real environments. The modular development is concluded, with the visualization and control modules implemented. *Scripts* for communication via ROSBridge are operational and integrated into the interface, and functionalities such as 2D mapping with LiDAR and the implementation of *waypoints* are working. Planned features for the first development phase of the undergraduate thesis, such as plotting 2D maps with point clouds and autonomous navigation by waypoints were implemented successfully, tests were also conducted to validate the positioning accuracy on

the map and the interface's effectiveness. The system demonstrates potential for applications in indoor and semi-structured environments, contributing to the efficient integration between graphical interfaces and robotic systems.

Index Terms—application, app, Autonomous robot, graphical interface, LiDAR, ROSBridge, Streamlit, web.

I. INTRODUÇÃO

A robótica autônoma tem se tornado cada vez mais presente em aplicações críticas que abrangem desde operações em ambientes extremos até tarefas cotidianas de automação. Estudos recentes na área destacam que a eficácia operacional desses sistemas depende fundamentalmente da integração harmoniosa entre interfaces de usuário bem projetadas e arquiteturas de controle confiáveis. Essa combinação estratégica não apenas possibilita uma interação mais intuitiva entre humanos e máquinas, mas também garante a execução autônoma eficiente de missões complexas, mantendo sempre a capacidade de supervisão e intervenção humana quando necessária [1].

Eu gosto de estudar robótica autônoma, e tenho interesse em desenvolver uma interface gráfica para controle de robôs autônomos, integrando-a com o ROS (Robot Operating System) para facilitar a comunicação e o controle em tempo real. O objetivo é criar uma plataforma que permita a visualização de dados sensoriais, a definição de metas de navegação e o controle manual do robô, utilizando tecnologias modernas de desenvolvimento de interfaces. A motivação para este projeto surge da necessidade de melhorar a interação entre humanos e robôs, tornando o controle mais acessível e eficiente, especialmente em ambientes internos onde a navegação autônoma pode ser desafiadora [2].

Estudos na área de robótica autônoma demonstram que os sistemas de controle para essas plataformas normalmente empregam técnicas consolidadas como controladores PID (proporcional integral derivativo), lógica *fuzzy* e redes neurais [3]. No âmbito das interfaces gráficas, observa-se uma diversidade de abordagens, desde soluções *desktop* baseadas em ROS até interfaces *mobile* para Android, cada uma com seus requisitos específicos de integração [4, 5]. Um desafio comum identificado na literatura é a dificuldade em estabelecer uma comunicação eficiente entre o núcleo de controle do robô e diferentes tipos de interfaces gráficas, frequentemente resultando em atrasos indesejados ou perda de sincronismo nos dados, independentemente da plataforma utilizada [2].

¹Trabalho de Conclusão de Curso apresentado ao Curso de Engenharia de Computação no segundo semestre de 2025, Área de Ciências Exatas e Tecnológicas da Universidade do Oeste de Santa Catarina como requisito parcial à obtenção do grau de bacharel em Engenharia de Computação.

A integração entre interfaces gráficas e sistemas de controle representa um dos principais desafios na robótica autônoma, impactando diretamente a eficácia operacional das plataformas robóticas [1, 2]. Este trabalho propõe uma solução que combina interfaces intuitivas com algoritmos de controle adaptativos, atendendo aos requisitos críticos de usabilidade e desempenho em sistemas robóticos autônomos.

A relevância desta pesquisa se manifesta em três aspectos principais: a otimização da interação humano-robô, essencial para sistemas de controle e supervisão; a contribuição para arquiteturas padronizadas, facilitando a replicabilidade de soluções; e a aplicação em cenários complexos, como navegação em ambientes dinâmicos e inspeção automatizada [2].

A modalidade *Trekking* em competições robóticas desafia os robôs a navegarem autonomamente em terrenos irregulares, superando obstáculos utilizando fusão de dados sensoriais (LIDAR, IMU e câmeras) para mapeamento 3D e tomada de decisão [3]. As regras avaliam não apenas a conclusão do percurso, mas também critérios como suavidade de movimento, eficiência energética e precisão de posicionamento (tolerâncias ≤ 5 cm). Essas regras tornam o ambiente de competição um *benchmark* ideal para validar sistemas robóticos integrados, já que exige interfaces gráficas responsivas, algoritmos de controle adaptativos e comunicação robusta, requisitos que demonstram transferibilidade direta para aplicações práticas como agricultura de precisão e inspeção de infraestruturas [3].

O objetivo central do meu trabalho consiste no desenvolvimento de uma plataforma robótica integrada, com dois focos específicos: implementação de uma interface gráfica otimizada para controle robótico e validação em ambientes característicos de aplicações robóticas [1, 2].

Partindo da plataforma robótica já adquirida para o projeto, o desenvolvimento seguirá uma abordagem prática em três etapas integradas. Na fase inicial, será realizada a caracterização detalhada do *hardware* disponível, mapeando suas capacidades e limitações para orientar o projeto da interface gráfica e dos algoritmos de controle. A implementação adotará uma arquitetura modular, com desenvolvimento paralelo e teste individual de cada subsistema (interface móvel, núcleo de controle e módulo de comunicação) seguindo padrões de interoperabilidade consagrados na área. A etapa final consistirá na integração progressiva e validação do sistema completo, com testes em condições reais de operação que aproveitem plenamente as capacidades da plataforma física disponível, assegurando robustez e desempenho adequados às aplicações planejadas.

Este artigo está estruturado em sete seções. Na seção I, foi introduzido o tema da robótica autônoma e os desafios na integração interface-controle, com os objetivos do trabalho. Na seção II, foram abordados os conceitos teóricos fundamentais sobre navegação autônoma, algoritmos de controle e protocolos de comunicação. Na seção III, foram examinadas pesquisas anteriores sobre sistemas similares, identificando lacunas a serem superadas e ideias a serem exploradas. Na seção IV, foi descrita a metodologia em três fases: caracterização do *hardware*, desenvolvimento modular e validação. Na seção V, foram mostrados os resultados dos testes de desempenho

do sistema. Na seção VI, foram apresentados os resultados finais em condições reais de operação. Na seção VII, foram destacadas as conclusões e sugestões para trabalhos futuros.

II. REFERENCIAL TEÓRICO

Neste capítulo são apresentadas as tecnologias: robótica autônoma, telemetria, mapeamento, planejamento de trajetória e API, que foram utilizadas para o desenvolvimento do trabalho, com o objetivo de familiarizar o leitor a ter uma leitura mais compreensiva do mesmo.

A. Robótica Autônoma

A robótica autônoma tem evoluído continuamente, tornando-se cada vez mais relevante no cenário tecnológico atual. A capacidade dos robôs autônomos substituírem humanos em atividades perigosas ou repetitivas justifica os investimentos na área. Competições e projetos educacionais desempenham papel fundamental na formação de profissionais e no estímulo ao interesse pela robótica [6].

Alguns dos principais desafios são navegação, planejamento de trajetória, localização e desvio de obstáculos [7]. Os sistemas de navegação de robôs móveis dependem do nível de abstração da representação do ambiente [8]. Para determinar com precisão a posição e a orientação do robô móvel, é essencial que o ambiente seja modelado em uma estrutura simples e compreensível. A parte desafiadora da localização é estimar a posição do robô e a orientação da qual essa informação pode ser obtida de sensores e outros sistemas. Para lidar com o desafio da localização, uma boa técnica deve ser desenvolvida para tratar erros, fatores de interferência, medições incorretas e estimativas imprecisas [9]. Desvio de obstáculos é uma tarefa essencial no campo da robótica, pois é fundamental que o robô alcance seu destino sem ser bloqueado por qualquer obstáculo ou sofrer colisões em seu trajeto [10]. Por esse motivo, algoritmos livres de colisão são um requisito indispensável para robôs móveis autônomos [1, 7].

Os robôs autônomos têm aplicações variadas: podem atuar desde o campo, monitorando lavouras, até ambientes industriais, seja atuando em processos produtivos como linhas de montagem e gestão logística de estoques em armazéns.

B. ROS

O Robot Operating System (ROS) é um conjunto de bibliotecas de *software* e ferramentas que auxiliam no desenvolvimento de aplicações robóticas. De *drivers* a algoritmos de ponta, e com poderosas ferramentas para desenvolvedores, o ROS oferece o que você precisa para o seu próximo projeto de robótica. E tudo isso é de código aberto [11].

O ROS vai além de ser apenas um conjunto de bibliotecas, ele funciona como um ecossistema robusto que padroniza a comunicação entre diferentes componentes de *software* em um robô. Ele oferece um modelo de programação distribuída, onde múltiplos nós (programas executáveis independentes) podem se comunicar enviando e recebendo mensagens através de tópicos. Isso permite que desenvolvedores de diversas áreas colaborem em um mesmo projeto robótico, com cada um focando em um módulo específico (como navegação, percepção ou controle), e o ROS garantindo a integração.

C. Telemetria

A palavra telemetria é a união de duas palavras gregas. *Tele* significa longe e *meter* significa medir. Por isso, telemetria significa realizar medições à distância, ou em local remoto. Os ingredientes essenciais de qualquer sistema de telemetria incluem pelo menos um sensor, uma antena transmissora de baixo ganho, uma antena receptora de alto ganho, um receptor e um mostrador [12].

Teste de veículos foi uma das primeiras aplicações da telemetria. Inicialmente, não tripulados, como mísseis guiados, requeriam telemetria, para que os engenheiros soubessem o que se passava dentro do míssil. Veículos tripulados também requeriam telemetria para medir parâmetros, tais como fadiga da asa, que não eram perceptíveis aos tripulantes [12].

D. Mapeamento e Planejamento de Trajetória

A técnica geral de *roadmap* para planejamento de trajetórias consiste em capturar a conectividade do espaço livre do ambiente em uma rede de curvas de uma dimensão, denominada *roadmap*. Uma vez que esta rede tenha sido obtida, ela é vista como um conjunto padrão de caminhos. O planejamento de trajetórias reduz-se então a conectar as posições iniciais e finais do robô ao *roadmap* e buscar neste um caminho entre esses dois pontos, conforme pode ser observado na Figura 1 [10].

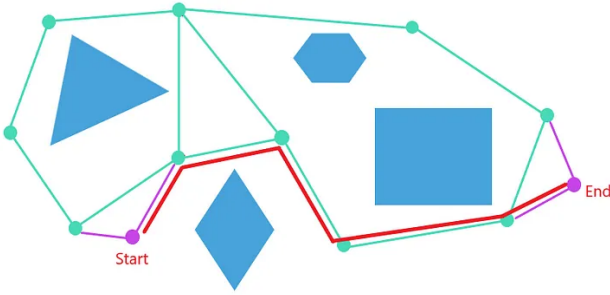


Fig. 1. Gráfico de navegação *roadmap* para encontrar a melhor rota.

Quando não há um mapa do ambiente obtido *a priori*, é preciso construir o mapa durante a navegação. Nesse caso, técnicas de localização e mapeamento simultâneo (SLAM) são usadas [13].

Resolver o problema de SLAM envolve a estimação da trajetória do robô e do mapa do ambiente enquanto o robô se desloca. Ao longo da movimentação do robô em poses sequenciais desconhecidas $x_{1:t} = x_1, x_2, \dots, x_t$ em um mapa desconhecido m , são obtidas uma sequência de odometria $u_{1:t} = u_1, u_2, \dots, u_t$ e observações do ambiente $z_{1:t} = z_1, z_2, \dots, z_t$. O problema SLAM requer a obtenção da distribuição de probabilidades dada pela Equação 1, considerando que x_0 é a posição inicial do robô [8].

O Cartographer é um sistema de SLAM eficiente que opera em dois níveis: localmente ajusta a pose do robô em submapas usando um *scan matcher* (Ceres), enquanto globalmente refina a localização combinando todos os submapas em um mapa maior [14, 15]. Sua principal vantagem é a flexibilidade, com suporte a múltiplos sensores e resoluções, e a eficiência computacional que permite operação em tempo real [16].

Existe uma dificuldade intrínseca em conciliar as medições de localização com a posição verdadeira do dispositivo após longos períodos de navegação. O desafio origina-se na imprecisão dos sensores de medidas de odometria (ex: sonares, lasers e câmeras) devido às limitações físicas e imprevisibilidades do sistema [9].

E. API

APIs (Interface de Programação de Aplicações) simplificam e aceleram o desenvolvimento de aplicativos e *software*, permitindo que os desenvolvedores integrem dados, serviços e recursos de outros aplicativos, em vez de desenvolvê-los do zero. As APIs também oferecem aos proprietários de aplicativos uma maneira simples e segura de disponibilizar os dados e as funções dos aplicativos para os departamentos da organização. Os proprietários de aplicativos também podem compartilhar ou comercializar dados e funções para parceiros de negócios ou terceiros [17].

As APIs permitem o compartilhamento apenas das informações necessárias, mantendo ocultos outros detalhes internos do sistema, o que auxilia na segurança do sistema. Os servidores ou dispositivos não precisam expor totalmente os dados: as APIs permitem o compartilhamento de pequenos pacotes de dados, relevantes para a solicitação específica [17].

Podem existir vulnerabilidades de segurança devido à implementação da API. Por exemplo, a falha em verificar o tamanho dos *inputs* na sua API pode permitir que um atacante derrube seu servidor ao enviar um *input* extremamente grande que consuma toda a memória disponível - um tipo de ataque de negação de serviço (DoS) [18].

ROSbridge é uma especificação de protocolo de rede em camada de aplicação para o ROS (*Robot Operating System*) que enfatiza o modelo cliente/servidor. Especificamente, o ROS-Bridge permite que clientes publiquem e escrevam mensagens em tópicos e invoquem serviços no ambiente de execução do servidor. O protocolo transporta mensagens formatadas em JSON por meio de *sockets* TCP ou *WebSockets* [19].

No trecho de código 1 a classe `roslibpy.ros` é instanciada com o IP do dispositivo ROS e também a porta padrão para comunicação WebSocket, e o método `client.run()` inicia a conexão com o servidor ROS.

Código 1: Configuração da Conexão WebSocket com ROS-Bridge

```
1 import roslibpy
2
3 # Configura a conexão WebSocket com o ROSbridge
4 rosbridge_ws_url = '192.168.137.150' # IP do
   dispositivo ROS
5 client = roslibpy.Ros(host=rosbridge_ws_url,
   port=9090)
6
7 # Conecta ao ROS
8 client.run()
```

F. Articulação das Tecnologias

As tecnologias apresentadas neste referencial teórico articulam-se de forma coerente e complementar na solução desenvolvida. A robótica autônoma fornece a base conceitual para o sistema, enquanto as técnicas de telemetria e os protocolos de comunicação (ROSbridge) garantem a troca

eficiente de dados entre os componentes. O SLAM e os algoritmos de planejamento de trajetória permitem a navegação autônoma, sendo acessíveis ao usuário através da interface desenvolvida com *frameworks* modernos. As APIs, por sua vez, atuam como elemento integrador, possibilitando a comunicação segura entre a aplicação e o sistema robótico. Esta sinergia tecnológica resulta em uma plataforma completa que aborda desde a percepção ambiental até a interação com o usuário final, cumprindo os objetivos propostos de forma integrada e eficiente.

III. TRABALHOS CORRELATOS

Lima e Silva (2022) [4] propuseram um sistema que viabiliza a programação e controle de um robô carro 2WD (duas rodas), utilizando Unity e C# para desenvolver o aplicativo. O aplicativo tem as funções de ser servidor para múltiplos dispositivos e fornecer controle manual sobre um robô. Essa proposta se assemelha ao projeto do meu trabalho ao utilizar um aplicativo para o controle do robô autônomo.

Zago (2018) [20] desenvolveu um sistema de telemetria sem fio para robôs de combate utilizando a tecnologia *Bluetooth* para comunicação e o sistema operacional Android com um aplicativo para monitoramento do robô. O sistema de telemetria foi desenvolvido de uma maneira que não afeta diretamente a funcionalidade, ou seja, o robô continua funcionando se o sistema de medição sofrer algum dano. Esse estudo se alinha a este trabalho pois utiliza a telemetria para comunicação com o robô e também utiliza um aplicativo para controle dos dados.

Werneck et al. [21] elaboraram um sistema de telemetria para uma embarcação movida a energia solar para monitorar dados elétricos e de desempenho do barco solar em tempo real. Utilizaram um ESP32 para processamento, LoRa para transmissão de dados a longas distâncias, .NET para desenvolvimento da interface e outras tecnologias. O sistema mede e transmite dados para uma estação terrestre via LoRa ou conexão 4G. Os dados são processados e exibidos em uma interface gráfica desenvolvida em .NET para monitoramento e análise. Este projeto é semelhante ao meu estudo por utilizar tecnologias de telemetria e interface gráfica para monitoramento em tempo real de parâmetros do sistema embarcado.

Pico et al. [5] propuseram um sistema de alarme e teleoperação em tempo real para falhas de navegação autônoma, comparando ROS 1 e ROS 2. O sistema integra uma interface web baseada em Vue.js e um *joystick* com *feedback* háptico para monitorar e controlar robôs em ambientes dinâmicos. A arquitetura utiliza sensores LiDAR (*Light Detection and Ranging*), câmeras RGB-D e IMU para detectar colisões iminentes ou instabilidades em terrenos irregulares, acionando alertas visuais e vibrações no *joystick*. Essa proposta se assemelha ao meu trabalho ao empregar uma interface gráfica para teleoperação e integração multimodal de sensores, com destaque para a comunicação via ROSBridge.

Perin et al. [3] desenvolveram um robô móvel autônomo para competições ao ar livre, utilizando um chassi comercial adaptado. O sistema de percepção do robô é composto por um sensor de orientação de 9 eixos e um encoder incremental para localização e odometria. Para o controle de trajetória e velocidade, foram implementados controladores PID em C++,

e o sistema foi simulado em um ambiente ROS integrado com a plataforma V-REP antes da implementação física. A proposta se assemelha ao meu trabalho ao empregar um sistema embarcado para o controle autônomo de um robô, com a particularidade de focar na navegação por coordenadas pré-definidas em campo aberto.

IV. METODOLOGIA

Este trabalho adota uma abordagem metodológica estruturada em três fases para desenvolver uma interface gráfica intuitiva, estabelecer uma conexão robusta e validar o sistema em ambientes operacionais reais, com foco em robôs autônomos em ambientes internos e semi-estruturados.

A metodologia inicia com a análise do *hardware* comercial disponível e a definição de requisitos técnicos. Em seguida, avança-se para o desenvolvimento paralelo dos subsistemas. Isso inclui a interface gráfica de usuário (GUI) com testes de usabilidade, a conexão robô-interface e os protocolos de comunicação via ROSBridge, conforme ilustrado na Figura 2. O foco desta etapa é priorizar a exibição em tempo real de parâmetros essenciais, como posição, orientação e estado do sistema. Dessa forma, não são incorporados recursos avançados de aprendizado de máquina ou análise preditiva. A etapa final consiste em testes de integração e validação em ambientes simulados (RViz) e reais, limitados pelo período de desenvolvimento do trabalho acadêmico, o que restringe a quantidade de cenários avaliados.

Essa abordagem sequencial assegura a verificação individual de cada componente antes da integração final, o que garante a eficiência operacional nas condições práticas previstas. Além disso, tal estrutura abre oportunidades para futuras expansões, como o desenvolvimento de componentes personalizados ou aplicações em ambientes externos.

A. Análise do Hardware

A primeira fase da metodologia caracteriza o NVIDIA Jetson Orin Nano Developer Kit, plataforma do sistema robótico. O kit inclui um módulo Jetson Orin Nano 8GB com GPU NVIDIA Ampère (1024 núcleos CUDA), CPU de 6 núcleos e 8GB de memória LPDDR5, oferecendo até 40 TOPS (*Tera Operations Per Second*) para aplicações de robótica. Sensores como LiDAR, IMU de 6 eixos e ultrassônicos, conectados via USB 3.2 Tipo-A ou conector de 40 pinos, suportam mapeamento 2D e navegação. Dois conectores MIPI CSI permitem câmeras para visão computacional. O armazenamento usa microSD ou SSD NVMe (M.2 Key-M). A comunicação ocorre via Wi-Fi, integrando com o ROS via ROSBridge [16].

B. Arquitetura do Sistema

O sistema robótico é projetado com uma arquitetura modular distribuída em três camadas principais, concebida para oferecer alta escalabilidade e flexibilidade na integração com diferentes plataformas robóticas e configurações de sensores. Na camada de percepção, os sensores atuam como os órgãos sensoriais do robô: um LiDAR para mapeamento 3D do ambiente, uma IMU de 6 eixos para medição inercial e sensores ultrassônicos para detecção de obstáculos próximos.

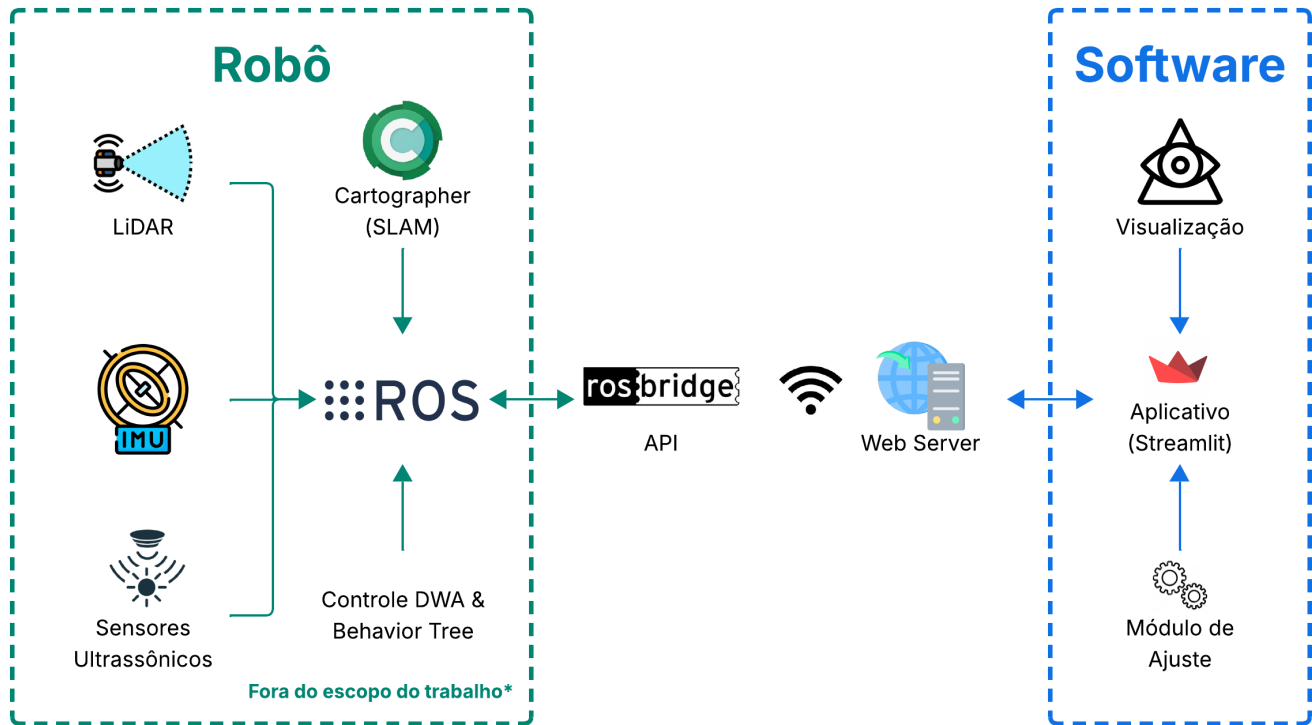


Fig. 2. Diagrama da arquitetura do sistema.

A camada de processamento reside em um computador embarcado que opera o ROS (*Robot Operating System*). Importante ressaltar que, apesar da nomenclatura, o ROS funciona como um *framework* de software para robótica, e não como um sistema operacional em si. Essa camada estabelece a comunicação com os demais componentes através do ROSBridge, gerenciando a telemetria (incluindo dados de velocidade e status do sistema) e a integração direta com a interface. A ampla utilização do ROS como padrão na indústria e academia contribui para a facilidade de integração com uma diversidade de plataformas robóticas e dispositivos, o que intrinsecamente melhora a escalabilidade da arquitetura.

Na camada de interface com o usuário, o Streamlit [22] fornece uma aplicação web acessível via navegador, exibindo dados de telemetria em tempo real e permitindo o controle manual quando necessário. A interface se conecta ao ROS através do ROSBridge, garantindo baixa latência na troca de comandos. Medidas de segurança como autenticação de usuário e criptografia serão implementadas para proteger a comunicação [22].

O fluxo de dados segue um ciclo contínuo: os sensores enviam informações para o ROS, que processa e toma decisões de navegação, enquanto a interface exibe os dados, recebe e envia comandos ao usuário. Os módulos são projetados para operar de forma independente em caso de falha pontual. A arquitetura desacoplada e a comunicação padronizada via ROS conferem ao sistema uma notável capacidade de adaptação e expansão, permitindo a integração com variados conjuntos de sensores e a fácil adição de novas funcionalidades.

O desenvolvimento da interface gráfica prioriza a criação de uma experiência intuitiva para o usuário final, com foco na implementação de um design responsivo que ofereça *feedback* visual imediato para todas as interações.

O módulo de comunicação é desenvolvido para garantir a troca de dados eficiente e confiável entre a interface e o robô. O protocolo ROSBridge é implementado para telemetria e integração com o sistema ROS, com atenção especial à redução de latência e manutenção da estabilidade da conexão.

O sistema foi desenvolvido utilizando tecnologias complementares que atendem a cada componente específico. Para a interface, foi adotada a biblioteca Streamlit [22], que permite criar aplicativos web interativos com Python [23], integrando-se diretamente ao núcleo do sistema ROS. Essa escolha elimina a necessidade de múltiplas linguagens e simplifica o desenvolvimento.

Também foram utilizadas outras duas bibliotecas Python: base64 para utilizar imagens no HTML e roslibpy[19] para comunicação da interface com o ROS.

A comunicação entre os módulos emprega o ROSBridge para telemetria geral e integração específica com o ROS. Essa combinação foi validada em trabalhos anteriores com requisitos semelhantes [5].

Para desenvolvimento e testes, o RViz[24] está sendo utilizado para simulações iniciais e o Ceres Solver[15] para otimização numérica no mapeamento. A arquitetura resultante, apoiada por essas tecnologias validadas na literatura, oferece robustez operacional e facilidade de manutenção.

A interface gráfica para o sistema de telemetria e para-

metrização do robô autônomo é projetada com o objetivo de proporcionar uma interação intuitiva e eficiente entre o usuário e o robô, assegurando a visualização de dados em tempo real, a configuração de parâmetros operacionais e o controle manual, quando necessário. Com base na arquitetura modular e nas tecnologias empregadas, como o Streamlit e o protocolo ROS-Bridge, a interface é estruturada em dois módulos: Visualização e Controle. Esses módulos foram definidos para atender aos requisitos de usabilidade, alinhando-se aos objetivos do projeto e às demandas de interação humano-robô.

O módulo de visualização apresenta, em tempo real, informações essenciais como velocidade, posição e mapeamento do ambiente do robô. Esses dados são exibidos por meio de indicadores numéricos, com um mapa 2D gerado pelo algoritmo SLAM (Cartographer)[14] para representar o terreno. Desenvolvida com Streamlit, a interface é responsiva, clara e acessível em dispositivos como navegadores web ou móveis, garantindo uma experiência amigável. A atualização dos dados ocorre com baixa latência, permitindo monitoramento contínuo.

O módulo de controle permite o usuário ajustar configurações operacionais, como limites de velocidade por meio de formulários interativos criados no Streamlit. Esses ajustes são enviados ao sistema via ROSBridge, garantindo uma comunicação rápida e segura. O módulo também oferece funcionalidades para operação manual utilizando setas direcionais para movimentação, essenciais em cenários que exigem intervenção humana, como testes ou situações de falha. A interface foi projetada para ser intuitiva, com elementos visuais que fornecem *feedback* imediato, como confirmações de comandos.

C. Critérios de Validação e Avaliação

O sistema será avaliado em três aspectos principais: desempenho, robustez e usabilidade.

Os testes serão inicialmente realizados em ambiente simulado (RViz/ROS) para validação dos algoritmos e arquitetura. Após aprovação na simulação, o sistema será testado em condições reais, para verificar a robustez e a segurança da operação. Todos os resultados serão comparados com os requisitos técnicos estabelecidos.

V. DESENVOLVIMENTO

A interface gráfica encontra-se concluída, tanto o componente visual (telas apresentadas na Seção VI), quanto as funcionalidades como o mapa 2D gerado pelo LiDAR dentro do ROS e a implementação de *waypoints* para navegação autônoma. As funcionalidades de controle estão operacionais.

A interface foi desenvolvida utilizando a biblioteca Streamlit, que proporcionou a estrutura, o funcionamento e a usabilidade das páginas. Adicionalmente, foi empregado CSS, aproveitando o suporte nativo do Streamlit a *markdown*, para realizar o posicionamento, redimensionamento e exibição de imagens, e também para estilização e responsabilidades dos botões para controle manual.

Os *scripts* destinados à comunicação com o ROS (*Robot Operating System*) foram integrados à interface e estão operacionais. Essa comunicação é estabelecida através de tópicos ROS, utilizando o paradigma *publisher/subscriber*. Por meio

do *script* de envio, que atua como um *publisher*, é possível ajustar a velocidade linear e angular do robô. Testes iniciais com o RViz demonstraram o sucesso na comunicação de parâmetros de velocidade, com os ajustes sendo visualizados em tempo real na simulação robótica. Já o *script* de requisição, funcionando como um *subscriber*, permite a obtenção de dados para atualização das informações exibidas na interface (em processo de desenvolvimento).

Para viabilizar a comunicação com o ROS, o *script* de requisição estabelece uma conexão via WebSocket utilizando o ROSBridge, configurando o endereço IP e a porta apropriados. Os testes de conexão e subscrição foram bem-sucedidos, permitindo o recebimento e visualização de qualquer tópico que está sendo utilizado pelo ROS naquele momento, incluindo os parâmetros de velocidade do robô via terminal. Após a conexão, este *script* atua como um *subscriber*, inscrevendo-se em tópicos determinados para receber dados relevantes do sistema robótico, conforme ilustrado no Código 2.

Código 2: Subscrição ao tópico /cmd_vel em ROS

```
1 # Subcrevendo ao tópico /cmd_vel
2 cmd_vel_subscriber = rospy.Topic(client,
3     '/cmd_vel', 'geometry_msgs/Twist')
4
5 # Conectando o callback para receber mensagens via
6     prompt de comando
7 cmd_vel_subscriber.subscribe(cmd_vel_callback)
```

Uma vez inscrito no tópico, os dados passam a ser exibidos no terminal.

O processo de envio de dados segue um procedimento análogo ao da requisição, mas com a interface atuando como um *publisher*. Neste cenário, a interface envia mensagens para tópicos ROS específicos, que são então consumidas pelo cliente ROS (neste caso, o robô virtual no RViz ou o robô físico). O Código 3 demonstra a estrutura para a publicação de dados no tópico *cmd_vel*.

Código 3: Código para publicação de dados em tópico ROS

```
1 # Cria um tópico /cmd_vel com tipo Twist
2 cmd_vel_publisher = rospy.Topic(client,
3     '/cmd_vel', 'geometry_msgs/Twist')
4
5 # Define velocidades linear e angular
6 linear_vel = 0.18421052631578947
7 angular_vel = 0.1578947368421052
8
9 # Cria a mensagem Twist
10 twist = {
11     'linear': {'x': linear_vel, 'y': 0.0, 'z':
12         0.0},
13     'angular': {'x': 0.0, 'y': 0.0, 'z':
14         angular_vel}
15 }
16
17 print(f'Received cmd_vel: Linear Velocity:
18     {linear_vel}, Angular Velocity: {angular_vel}')
```

Os arquivos responsáveis pelo processamento e tratamento dos dados recebidos via terminal encontram-se testados e finalizados.

VI. RESULTADOS

Na Figura A.1, referente ao módulo de visualização da interface, é apresentado o mapa 2D junto com a tecnologia do LiDAR para identificação de obstáculos, informações sobre a velocidade, posição e rotação do robô atualizados em tempo

real. Ao lado esquerdo existe uma barra lateral que permite a configuração do IP e da porta para conexão com o sistema rodando ROS, um *switch* para alterar o estado da atualização de parâmetros em tempo real (ligada/desligada), e um componente para navegar entre as telas da interface. Ao clicar em qualquer ponto do mapa um *waypoint* é definido, ao mesmo tempo, um círculo verde é desenhado em um segundo mapa para *feedback* visual de onde foi definido o *waypoint*, conforme mostra a Figura A.2, e as coordenadas são salvas ficando prontas para o envio ao ROS, feito pelo botão “Enviar *waypoint*”.

A Figura A.3 mostra um sistema Linux rodando o *framework* ROS, com dois terminais e o software RViz na tela. No terminal localizado no canto superior esquerdo, mostra o tópico `/goal_pose` após ter recebido o *waypoint* definido na interface. Ao receber a atualização no tópico, o ROS envia os dados ao RViz e o robô dentro da simulação inicia o trajeto até o destino. O terminal no canto inferior esquerdo mostra o tópico `/cmd_vel` com o ROS publicando a velocidade do robô em tempo real. Nesse modo de navegação por *waypoint* a velocidade é definida automaticamente pelo ROS. E o *software* à direita da imagem é o RViz utilizado para testar as funcionalidades da interface.

A Figura A.4 representa o módulo de controle da interface, exibindo uma tela com um interruptor para ativar o controle manual e uma seção de controle direcional para gerenciar os movimentos do robô. Adicionalmente, são apresentados campos interativos para a configuração de parâmetros, tais como a velocidade máxima linear e angular.

A Figura A.5 mostra o *software* RViz e um terminal com o tópico `/cmd_vel` sendo atualizado pela interface de controle. Os únicos objetos utilizados no tópico são o *x* da seção linear e o *z* da seção angular. Os controles para mover para frente e para trás utilizam apenas o *x* para se movimentar: para frente o valor é positivo e para trás negativo. Para fazer curvas, a velocidade angular e linear é a mesma, para virar para a esquerda os valores são iguais, porém para a direita o valor de *z* é negativo. As velocidades enviadas ao tópico `/cmd_vel` podem ser definidas via interface no módulo de controle, nos dois *sliders* localizados na parte inferior da tela mostrada na figura A.4: o da esquerda define a velocidade máxima linear e o da direita define a velocidade máxima angular.

O sistema foi submetido a testes de campo durante uma competição, operando sob forte interferência eletromagnética na frequência de 5 GHz. A interface demonstrou estabilidade na transmissão de dados, permitindo, sem latência perceptível, o envio de metas de navegação (*waypoints*), a atualização visual do mapa de ocupação e o controle direto dos atuadores do robô.

VII. CONSIDERAÇÕES FINAIS

Este trabalho teve como objetivo principal o desenvolvimento de uma interface gráfica para telemetria e parametrização de robôs autônomos, integrada ao ROS via ROSBridge, com foco em usabilidade e comunicação em tempo real, servindo como um modelo de integração real entre interface e sistema robótico. Os testes realizados demonstram a viabilidade e eficácia do sistema, com a implementação bem-sucedida

de funcionalidades como o mapeamento 2D com LiDAR e a gestão de *waypoints*.

A interface facilita a interação humano-robô em ambientes internos, utilizando Streamlit e ROSBridge para garantir uma visualização intuitiva dos dados e uma comunicação eficiente.

Na segunda fase de desenvolvimento do TCC, o objetivo foi expandir as capacidades da interface, implementando a gestão de *waypoints*, a plotagem do mapa em tempo real dentro do módulo de visualização, a integração com a nuvem de pontos do LiDAR, e a validação do projeto através da participação em uma competição de robótica.

O trabalho oferece uma contribuição significativa ao demonstrar a possibilidade de uma integração eficaz e escalável com sistemas robóticos, com potencial para avanços substanciais na robótica autônoma, dependendo da continuidade do desenvolvimento e aprofundamento dos testes.

A biblioteca Streamlit, embora ágil para prototipagem rápida, apresentou certas restrições em funcionalidades específicas, como a combinação de markdown com a formatação própria do Streamlit, onde as configurações de um sobrepõem as do outro sem um comportamento padronizado. Também limitou o desenvolvimento do trabalho no módulo de controle, onde a criação de botões personalizados e a disposição dos mesmos na interface não puderam ser totalmente customizadas, restringindo a experiência do usuário. Futuras versões da biblioteca podem vir a solucionar essas limitações, ampliando as possibilidades de desenvolvimento de interfaces mais sofisticadas.

Como propostas para trabalhos futuros, é possível aprimorar o módulo de controle com a implementação de uma funcionalidade de controle manual via teclado, incluindo suporte a teclas simultâneas para uma operação mais intuitiva. Adicionalmente, a navegação autônoma pode ser expandida para permitir a criação de rotas complexas, informando múltiplos *waypoints* sequenciais que o robô deve percorrer, incrementando a função atual de navegação por ponto único. Por fim, outra melhoria consiste na refatoração da plotagem do mapa para unificar a definição e a visualização de *waypoints* em um único componente visual, otimizando a interface do usuário ao remover a necessidade de um mapa secundário para *feedback*.

REFERÊNCIAS

- [1] Roland Siegwart, Illah Reza Nourbakhsh e Davide Scaramuzza. *Introduction to Autonomous Robots*. 2nd. Cambridge, MA: MIT Press, 2020.
- [2] Morgan Quigley, Brian Gerkey e William D. Smart. *Programming Robots with ROS: A Practical Introduction to the Robot Operating System*. Disponível também em formato eletrônico. Sebastopol, CA: O'Reilly Media, 2015. ISBN: 978-1-4493-2389-9.
- [3] Ana Carina Coninch Perin, Renato Gregolon Scortegagna, Kleyton Hoffmann, João Alberto Fabro, André Schneider de Oliveira e Marconi Januário. “Simulation and Implementation of an Autonomous Mobile Robot for Outdoor Competitions”. Em: *2020 IEEE International Conference on Industrial Technology (ICIT)*. 2020, pp. 395–400. DOI: 10.1109/ICIT45562.2020.9067268.

- [4] Lima D. I. A. Silva P. H. L. *Um sistema para programação e controle remoto de um robô 2WD usando um aplicativo para android*. 2022. URL: <https://repositorio.ufersa.edu.br/handle/prefix/8663>.
- [5] Nabih Pico, Giovanny Mite, Daniel Morán, Manuel S Alvarez-Alvarado, Eugene Auh e Hyungpil Moon. “Web-Based Real-Time Alarm and Teleoperation System for Autonomous Navigation Failures Using ROS 1 and ROS 2”. Em: *Actuators*. Vol. 14. 4. MDPI. 2025, p. 164.
- [6] Gabriel Niederauer, João Scherer Cipriani e Diego Carriho. “Desenvolvimento de um Robô Autônomo para Competições de Sumô Robótico”. Em: nov. de 2017.
- [7] Mary B Alatise e Gerhard P Hancke. “A review on challenges of autonomous mobile robot and sensor fusion methods”. Em: *IEEE access* 8 (2020), pp. 39830–39846.
- [8] Henrique Scharlau Coelho. “Mapeamento de ambiente para navegação de um robô móvel”. Em: (2024).
- [9] Felipe Jordão Freitas de Mendonça. “Proposta de Algoritmo de Localização por Meio de Odometria LiDAR”. Trabalho de Conclusão de Curso (Bacharelado em Engenharia de Computação). Diss. de mistr. Pato Branco, PR: Universidade Tecnológica Federal do Paraná (UTFPR), 2023. URL: <https://riut.utfpr.edu.br/jspui/bitstream/1/33270/1/propostaalgoritmolocalizacaoodometria.pdf>.
- [10] G d L Ottoni. “Planejamento de trajetórias para robôs móveis”. Em: *Projeto de Graduação em Engenharia de Computação–FURG, Rio Grande* (2000).
- [11] Open Robotics. *Robot Operating System (ROS)*. . Acessado em: 17 de julho de 2025. 2025.
- [12] A.N. de Mattos. *Telemetria e conceitos relacionados*. Alessandro Nicoli de Mattos. URL: <https://books.google.com.br/books?id=6sLBOUMaFrMC>.
- [13] Sebastian Thrun, Wolfram Burgard e Dieter Fox. *Probabilistic Robotics*. Cambridge, MA: MIT Press, 2005. ISBN: 9780262201629.
- [14] Abdurahman Dwijotomo, Mohd Azizi Abdul Rahman, Mohd Hatta Mohammed Ariff, Hairi Zamzuri e Wan Muhd Hafeez Wan Azree. “Cartographer slam method for optimization with an adaptive multi-distance scan scheduler”. Em: *Applied Sciences* 10.1 (2020), p. 347.
- [15] Sameer Agarwal, Keir Mierle e The Ceres Solver Team. *Ceres Solver*. Versão 2.2. Out. de 2023. URL: <https://github.com/ceres-solver/ceres-solver>.
- [16] NVIDIA. *Get Started with the Jetson Orin Nano Developer Kit*. <https://developer.nvidia.com/embedded/learn/get-started-jetson-orin-nano-devkit>. Acessado em: 28 de julho de 2025. 2025.
- [17] IBM. *O que é API?* Português. 2024. URL: <https://www.ibm.com/br-pt/topics/api> (acesso em 28/03/2025).
- [18] Neil Madden. *API security in action*. Simon e Schuster, 2020.
- [19] Czech Technical University in Prague. *Web Interface Report*. Rel. técn. Relatório técnico. Disponível em: https://cw.fel.cvut.cz/wiki/_media/misc/projects/nifti/sw/web_interface-report.pdf. Czech Technical University in Prague (CTU), Acesso em: [insira o ano, se disponível]. URL: https://cw.fel.cvut.cz/wiki/_media/misc/projects/nifti/sw/web_interface-report.pdf (acesso em 2025).
- [20] Ricardo de Moura Zago. “TELEMETRIA PARA ROBÔS DE COMBATE”. Em: (). URL: <https://www.ufsm.br/app/uploads/sites/495/2018/12/Ricardo-Zago.pdf>.
- [21] Adriano Werneck, Alex Candido, Evelyn Galdino, Alexandre Bitencourt e Daniel Dias. “DESENVOLVIMENTO DE UM SISTEMA DE TELEMETRIA APLICADO A UMA EMBARCAÇÃO SOLAR”. Em: (). URL: <https://desafiosolar.com.br/wp-content/uploads/2023/09/Arariboia-Artigo-Cientifico.pdf>.
- [22] Streamlit. *Streamlit • A faster way to build and share data apps*. <https://streamlit.io>. Acessado em: 28 de julho de 2025. 2025.
- [23] Python Software Foundation. *Python Language Reference, version 3.x*. 2023. URL: <https://www.python.org>.
- [24] Open Robotics. *RViz: ROS 3D Robot Visualizer*. 2025. URL: <http://wiki.ros.org/rviz>.

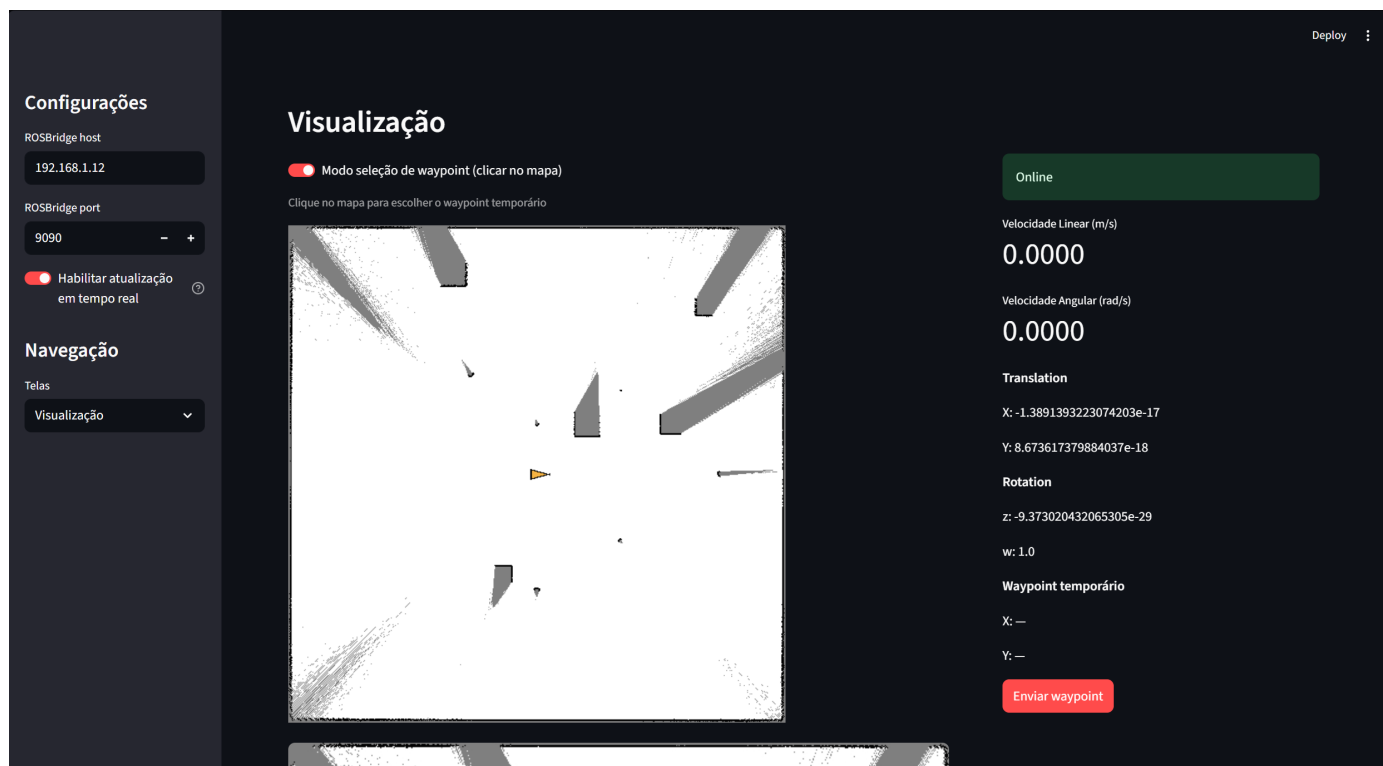


Fig. A.1. Módulo de visualização da interface.

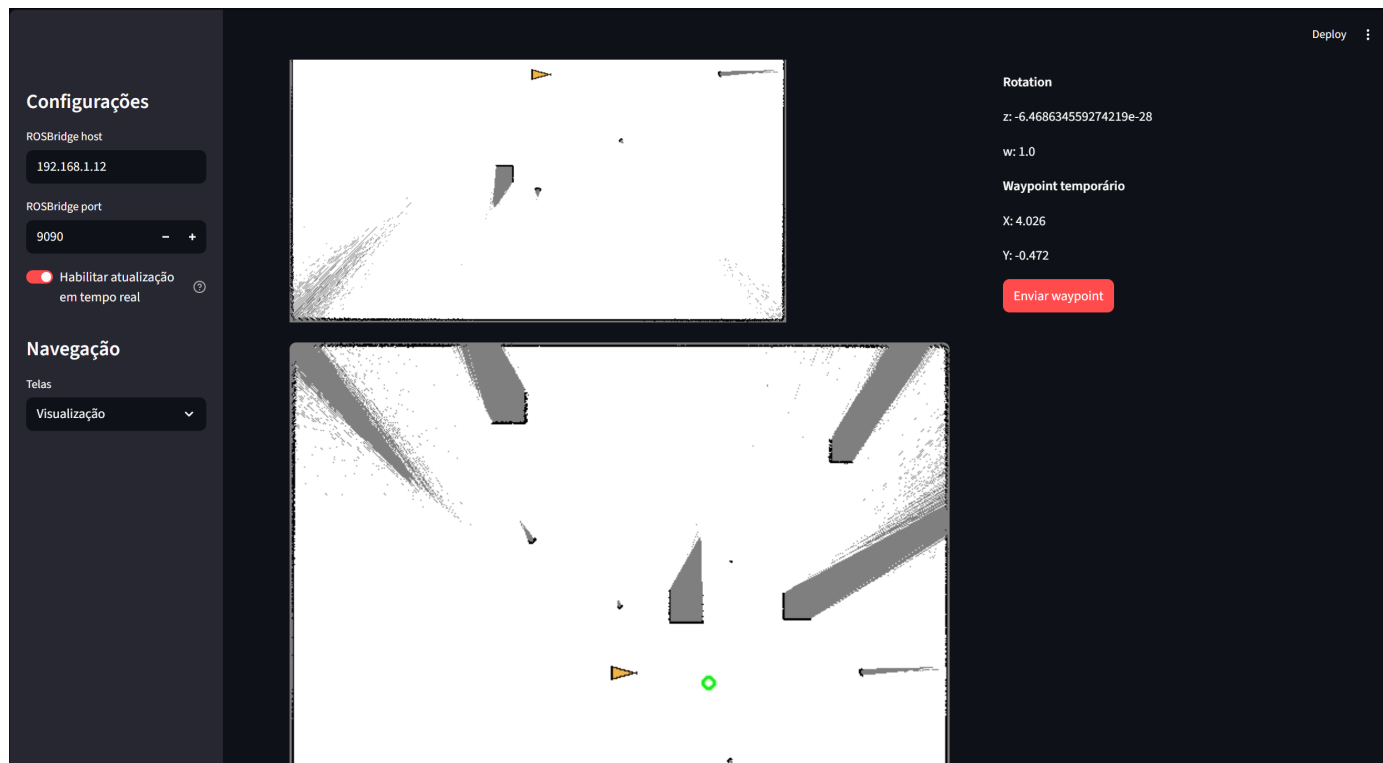


Fig. A.2. Funcionalidade de definição de waypoint na interface.

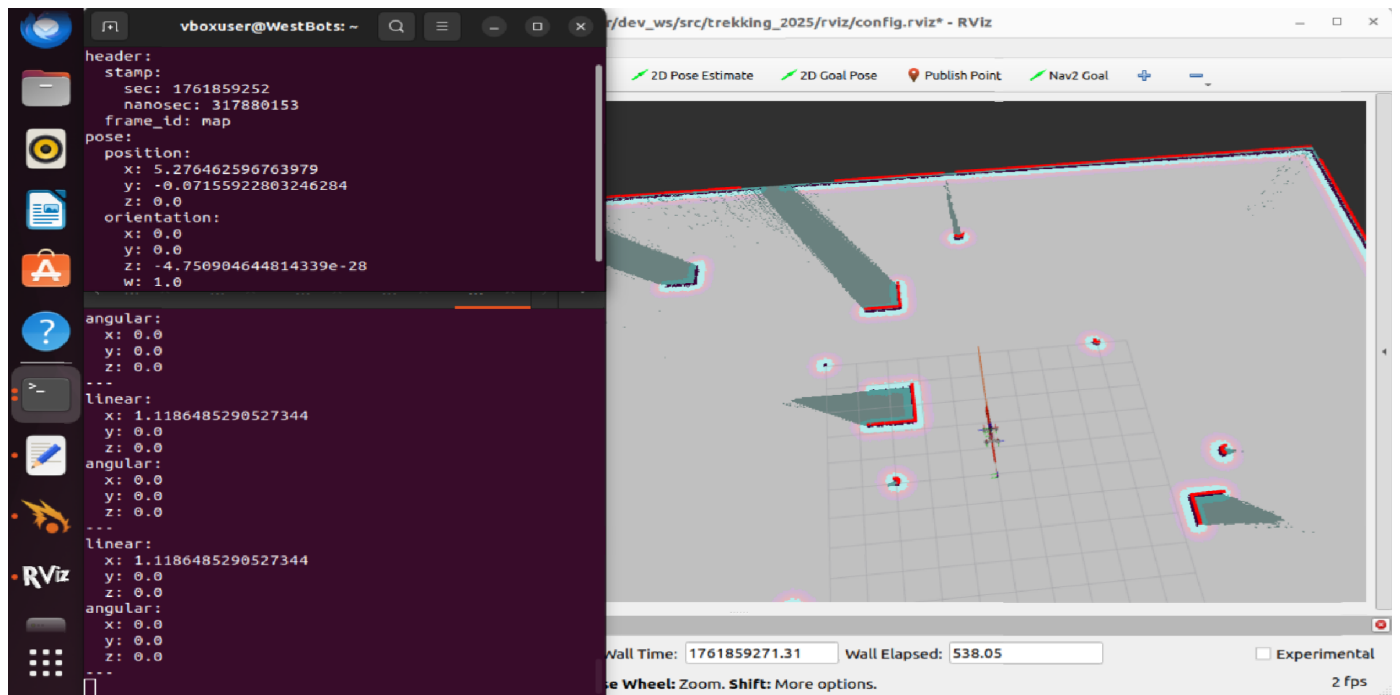


Fig. A.3. Envio de waypoint para o ROS e navegação automática do robô.

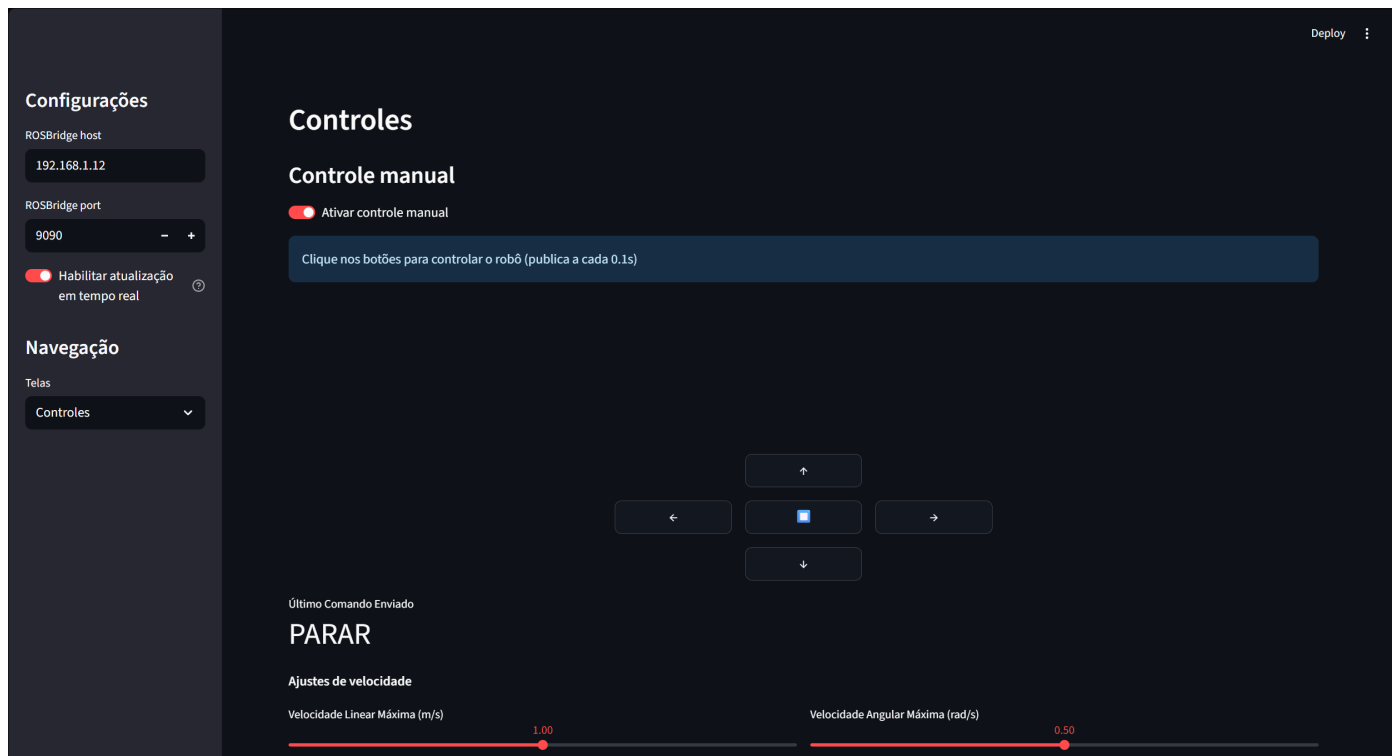


Fig. A.4. Módulo de controle da interface.

